

Docker + PostgreSQL + Alembic Cheatsheet

A quick-reference guide for every backend project — setup from scratch, plus the day-to-day commands.

Part 1 — Setting Up Docker From Scratch

This is the reusable setup you can follow at the start of every project that needs a database + backend running in containers.

1. Install Docker

Install Docker Desktop (Mac/Windows) or Docker Engine (Linux). This gives you the **docker** and **docker compose** commands. Verify with:

```
docker --version
docker compose version
```

2. Create your project folder

Structure your project so Docker knows what to build:

```
myproject/
  backend/
    app/
      requirements.txt
      Dockerfile
    docker-compose.yml
  .env
```

3. Write a Dockerfile for your backend

This tells Docker how to build the image for your FastAPI/Python app:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. Write docker-compose.yml

This defines and connects your services (backend + database) as one unit:

```
version: "3.9"

services:
  db:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
```

```
    - "5432:5432"
  volumes:
    - pgdata:/var/lib/postgresql/data

  backend:
    build: ./backend
    restart: always
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db

  volumes:
    pgdata:
```

5. Create a .env file

Keep secrets and connection info out of your code:

```
DATABASE_URL=postgresql+asyncpg://myuser:mypassword@db:5432/mydb
```

6. Build and start everything

This builds your images and starts the containers in the background:

```
docker compose up --build -d
```

7. Check that it's running

Confirm both containers (db + backend) are up:

```
docker compose ps
```

8. Set up Alembic inside the backend

Initialize migrations once, so your DB schema is version-controlled (see Part 3 below):

```
docker compose exec backend alembic init alembic
```

9. You're ready to build

From here on, you only need the day-to-day commands in Parts 2–4 below — you won't need to repeat this setup unless starting a brand-new project.

Part 2 — Docker Commands

The commands you'll reach for constantly once your containers exist.

Starting & Stopping

Command	Explanation
<code>docker compose up -d</code>	Start all services in the background (detached mode).
<code>docker compose up --build -d</code>	Rebuild images first, then start — use after changing the Dockerfile or dependencies.
<code>docker compose down</code>	Stop and remove all containers (keeps your data volume).
<code>docker compose down -v</code>	Stop containers AND delete volumes — wipes your database data too.
<code>docker compose restart</code>	Restart all running services without rebuilding.
<code>docker compose stop</code>	Stop containers without removing them.

Inspecting & Debugging

Command	Explanation
<code>docker compose ps</code>	List running containers for this project and their status.
<code>docker compose logs</code>	Show logs from all services.
<code>docker compose logs -f backend</code>	Follow (live-tail) logs from just the backend service.
<code>docker ps</code>	List ALL running containers on your machine (any project).
<code>docker ps -a</code>	List all containers, including stopped ones.
<code>docker images</code>	List all downloaded/built images.

Working Inside Containers

Command	Explanation
<code>docker compose exec backend bash</code>	Open an interactive shell inside the running backend container.
<code>docker compose exec backend python</code>	Open a Python shell inside the backend container.
<code>docker compose run --rm backend <cmd></code>	Spin up a temporary container to run one command, then remove it (use when the service isn't already running).

Cleaning Up

Command	Explanation
<code>docker compose down --rmi all</code>	Stop containers and remove the images built for this project.
<code>docker system prune</code>	Remove unused containers, networks, and dangling images (frees disk space).

Command	Explanation
<code>docker volume ls</code>	List all volumes (where persistent data like your DB lives).
<code>docker volume rm <name></code>	Delete a specific volume.

Part 3 — PostgreSQL Commands Inside Docker

These run against the Postgres container directly, using **psql** (Postgres's command-line client).

Connecting

Command	Explanation
<code>docker compose exec db psql -U myuser -d mydb</code>	Open an interactive psql session inside the db container.
<code>docker compose exec db psql -U myuser -l</code>	List all databases on the Postgres server.

Inside the psql prompt

Command	Explanation
<code>\l</code>	List all databases.
<code>\c mydb</code>	Connect to (switch to) a specific database.
<code>\dt</code>	List all tables in the current database.
<code>\d tablename</code>	Describe a table's columns, types, and constraints.
<code>\du</code>	List all users/roles.
<code>\q</code>	Quit the psql session.
<code>SELECT * FROM tablename;</code>	Run a plain SQL query, same as anywhere else.

Backup & Restore

Command	Explanation
<code>docker compose exec db pg_dump -U myuser mydb > backup.sql</code>	Dump the entire database to a SQL file on your host machine.
<code>cat backup.sql docker compose exec -T db psql -U myuser -d mydb</code>	Restore a database from a SQL dump file.

Tip: your Postgres data lives in a Docker volume (see `docker-compose.yml` above). As long as you don't run **docker compose down -v**, your data survives container restarts.

Part 4 — Alembic Commands

Alembic manages your database schema as version-controlled migrations. All commands below run through Docker, from your project root.

Setup (once per project)

Command	Explanation
<code>docker compose exec backend alembic init alembic</code>	Creates the alembic/ folder and alembic.ini config file.

Everyday Migration Commands

Command	Explanation
<code>docker compose exec backend alembic revision --autogenerate -m "msg"</code>	Auto-generates a migration by comparing your models to the current DB schema.
<code>docker compose exec backend alembic revision -m "msg"</code>	Creates a blank migration file for manual/data edits (no autogenerate).
<code>docker compose exec backend alembic upgrade head</code>	Applies all pending migrations, bringing the DB up to the latest version.
<code>docker compose exec backend alembic upgrade +1</code>	Applies just the next one migration.
<code>docker compose exec backend alembic downgrade -1</code>	Rolls back the most recent migration by one step.
<code>docker compose exec backend alembic downgrade base</code>	Rolls back ALL migrations — empties the schema.

Checking Status

Command	Explanation
<code>docker compose exec backend alembic current</code>	Shows which migration version the DB is currently on.
<code>docker compose exec backend alembic history</code>	Lists all migrations in order, oldest to newest.
<code>docker compose exec backend alembic history --verbose</code>	Same as above, with full detail per migration.
<code>docker compose exec backend alembic show <revision></code>	Shows the up/down logic of one specific migration.
<code>docker compose exec backend alembic stamp head</code>	Marks the DB as up-to-date WITHOUT running migrations (schema already matches).

Common Gotchas

- `--autogenerate` detects new tables/columns/indexes, but usually misses column type changes, renames, and some constraint changes — always review the generated file before applying it.

- If containers aren't running yet, use **docker compose run --rm backend alembic ...** instead of **exec** — **run** spins up a temporary container, **exec** needs the service already up.
- With async SQLAlchemy, your **alembic/env.py** needs the async engine wired up correctly, or autogenerate silently detects nothing even when your models changed.
- Replace **backend / db** above with whatever your services are actually named in docker-compose.yml (check with **docker compose ps**).

Keep this as your go-to reference for every new project's backend setup.